MISP modules

Enrichment, data exchange and workflow actions

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 29 October 2026

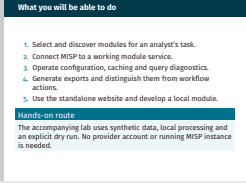
MISP modules
Enrichment, data exchange and workflow actions
CIRCL / MISP Project
https://www.misp-project.org/
Reference snapshot: 29 October 2026

What you will be able to do

1. Select and discover modules for an analyst's task.
2. Connect MISP to a working module service.
3. Operate configuration, caching and query diagnostics.
4. Generate exports and distinguish them from workflow actions.
5. Use the standalone website and develop a local module.

Hands-on route

The accompanying lab uses synthetic data, local processing and an explicit dry run. No provider account or running MISP instance is needed.



Use this as a 60–90 minute session, extending the optional development lab. Ask which roles are present: analyst, platform operator, integration developer. The README provides shorter teaching routes and expected exercise outcomes.

A dated snapshot, not a promise about every install

- Reviewed against upstream `main`, commit `3a903305cf37`, on 29 October 2026.
- Package metadata declares version `3.1.1`. Features on this branch can postdate a published package.
- Source inventory: **270 candidate module files** across four families.
- Installed dependencies, credentials and service reachability determine which integrations can actually run.

Inspect the running service

Check `/version`, `/modules` and `/openapi.json`. A source file count is not the number of operational integrations.

Pinned source links and the inventory method are in this deck's README.



The count excludes __init__.py and underscore-prefixed files. Do not present 3.1.1 as guaranteeing every feature in this training. Recheck the revision before teaching a later release.

Four families, different contracts

Family	Directory	Files	Purpose
Expansion	`expansion`	203	Enrich an observable; some modules also support hover.
Import	`import_mod`	22	Parse uploaded data or retrieve a report into MISP results.
Export	`export_mod`	32	Transform supplied MISP data into a downloadable format.
Action	`action_mod`	13	Run an operation from a workflow, such as a notification or publication.

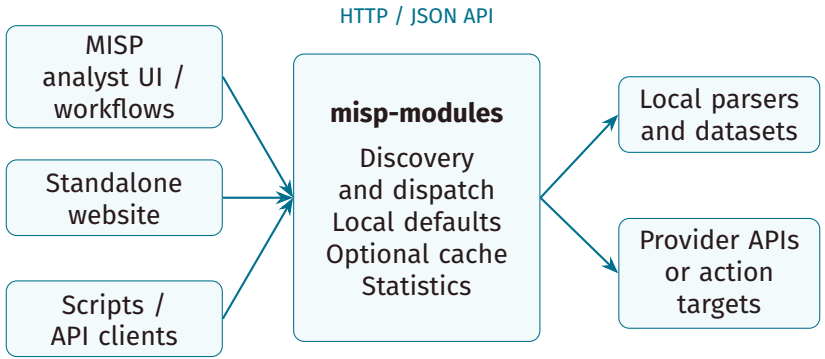
Directory names are loader families. Metadata uses `expansion/hover`, `import`, `export` and `action`.

Four families, different contracts

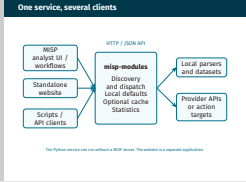
Family Directory Files Purpose
Expansion expansion 203 Enrich an observable; some modules also support hover.
Import import_mod 22 Parse uploaded data or retrieve a report into MISP results.
Export export_mod 32 Transform supplied MISP data into a downloadable format.
Action action_mod 13 Run an operation from a workflow, such as a notification or publication.
Directory names are loader families. Metadata uses expansion/hover, import, export and action.

These are the four ModuleType values in misp-modules, not every integration family recognized by MISP core. Cortex and other core integrations are separate. Hover is a capability of expansion modules, not a fifth directory.

One service, several clients



The Python service can run without a MISP server. The website is a separate application.



The service can be co-located with MISP or run elsewhere. Distinguish MISP's event database, the module process, and the website's own database/session history. They are not a shared persistence layer. Some operations are local; many enrichments contact third parties.

Use and understand modules

Choose a module by the question you need to answer

Analyst question	Examples in the current source
What does this URL contain?	`extract_url_components`: local structured parsing.
What infrastructure is related?	`dns`, `reversedns`, `circl_passivedns`: different DNS evidence.
Is this observable known or noisy?	`hygiene`, `hashlookup`, `greynoise`.
What vulnerability context exists?	`cve`, `gcve_cpe`, `rapid7_attackerkb`.
What does analysis tooling know?	`mwdb_query`, `assemblyline_query`, sandbox query/submit integrations.

Read the module's input types, configuration, dependencies and side effects. A lookup and a submission are different operational choices.

Choose a module by the question you need to answer

Analyst question | Examples in the current source
What does this URL contain? | extract_url_components: local structured parsing.
What infrastructure is related? | dns, reversedns, circl_passivedns: different DNS evidence.
Is this observable known or noisy? | hygiene, hashlookup, greynoise.
What vulnerability context exists? | cve, gcve_cpe, rapid7_attackerkb.
What does analysis tooling know? | mwdb_query, assemblyline_query, sandbox query/submit integrations.
Read the module's input types, configuration, dependencies and side effects. A lookup and a submission are different operational choices.

Examples are representative, not an endorsement or a complete catalog. Hygiene adds local warninglist context; a match is not proof of benignness. GCVE CPE matches a catalog record and does not establish that a product is vulnerable. Provider coverage depends on API version, account entitlement and quotas.

Discover what is really loaded

```
curl --fail --silent --show-error \
  http://127.0.0.1:6666/modules |
  jq '.[] | {name, type, mispattributes, meta}'
```

- `name`: API identifier, usually the Python filename.
- `meta.name`: human-readable display name, when supplied.
- `mispattributes`: input/output capabilities and format.
- `meta.config`: expected settings, not configured secret values.
- `meta.module-type`: capabilities used by MISP.

Discover what is really loaded
curl --fail --silent --show-error \
http://127.0.0.1:6666/modules |
jq '.[] | {name, type, mispattributes, meta}'
name: API identifier, usually the Python filename.
meta.name: human-readable display name, when supplied.
mispattributes: input/output capabilities and format.
meta.config: expected settings, not configured secret values.
meta.module-type: capabilities used by MISP.

A missing optional library can cause the loader to skip a module. The metadata requirements field is descriptive; it does not install libraries. Restart after code changes; module loading and OpenAPI generation happen at startup. Teach participants to select by API identifier rather than display name.

Two expansion formats still coexist

Simplified / legacy

```
{"module": "training_uppercase",
 "text": "misp training"}

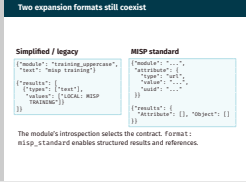
{"results": [
  {"types": ["text"],
   "values": ["LOCAL: MISP
    TRAINING"]}
]}
```

MISP standard

```
{"module": "...",
 "attribute": {
   "type": "url",
   "value": "...",
   "uuid": "..."
 }}

{"results": {
  "Attribute": [], "Object": []
}}
```

The module's introspection selects the contract. `format: misp_standard` enables structured results and references.



The ellipses on the right describe the shape, not a runnable payload. The next slide supplies a complete example. Do not send one universal request shape to every module. Some structured modules also return Tag data. The left example uses the lab prefix defaults and the legacy result format.

A complete local enrichment request

Save or use the included `examples/url-request.json`:

```
{
  "module": "extract_url_components",
  "attribute": {
    "type": "url",
    "value": "https://portal.example.org/a?item=1",
    "uuid": "13d8d18b-784a-42a0-9a58-a9cddbfda561"
  }
}
```

```
curl --fail --silent --show-error \
  http://127.0.0.1:6666/query \
  -H 'Content-Type: application/json' \
  --data @examples/url-request.json
```



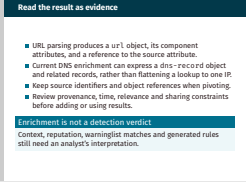

Run from this deck's directory. This module parses the URL locally: it does not fetch portal.example.org. The UUID is a synthetic source attribute. Show the complete response, then find results.Object and the url object. The lab verifies the domain, host, resource path and source reference.

Read the result as evidence

- URL parsing produces a `url` object, its component attributes, and a reference to the source attribute.
- Current DNS enrichment can express a `dns-record` object and related records, rather than flattening a lookup to one IP.
- Keep source identifiers and object references when pivoting.
- Review provenance, time, relevance and sharing constraints before adding or using results.

Enrichment is not a detection verdict

Context, reputation, warninglist matches and generated rules still need an analyst's interpretation.



Avoid the old hardcoded DNS answer or a claim that all domains map to one address. An observed DNS answer is time-bound; passive DNS and active resolution answer different questions. Review detection flags before promoting enrichment into a downstream control.

Connect MISP: service discovery and user access

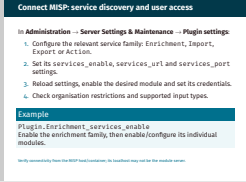
In **Administration** → **Server Settings & Maintenance** → **Plugin settings**:

1. Configure the relevant service family: `Enrichment`, `Import`, `Export` or `Action`.
2. Set its `services_enable`, `services_url` and `services_port` settings.
3. Reload settings, enable the desired module and set its credentials.
4. Check organisation restrictions and supported input types.

Example

`Plugin.Enrichment_services_enable`
Enable the enrichment family, then enable/configure its individual modules.

Verify connectivity from the MISP host/container; its localhost may not be the module server.



Enabling Enrichment does not enable Import, Export or Action. The current MISP Module model checks each family's enable switch and builds its URL. Action module parameters and workflow behavior are module-specific. Distinguish service startup, discovery, MISP enablement and user permissions: each is a separate checkpoint.

Deploy and operate the service

Choose the dependency footprint

Install	Python command	What to expect
Base	`pip install .`	Core service, workflow support and modules whose imports are satisfied.
Common extras	`pip install '.[minimal]'`	A broader set of Python dependencies, intended without extra system packages.
Broad extras	`pip install '.[all]'`	Wider coverage; some native tools/libraries are also required.

The snapshot requires **Python 3.10 or newer, below 4.0**. The lab uses Python 3.12 and the base installation.

Choose the dependency footprint

Install | Python command | What to expect
Base | pip install . | Core service, workflow support and modules whose imports are satisfied.
Common extras | pip install '.[minimal]' | A broader set of Python dependencies, intended without extra system packages.
Broad extras | pip install '.[all]' | Wider coverage; some native tools/libraries are also required.

The snapshot requires **Python 3.10 or newer, below 4.0**. The lab uses Python 3.12 and the base installation.

Run these commands from a source checkout inside a virtual environment. For a published package the equivalents are misp-modules, misp-modules[minimal], and misp-modules[all]. Shell-quote extras. Native examples include Poppler, zbar and Tesseract. Installing all extras does not create provider accounts, API entitlements or valid credentials. A missing-yara warning is a heuristic for reduced dependencies, not proof that every other module failed.

Use a virtual environment and a supervised process

```
git clone https://github.com/MISP/misp-modules.git
cd misp-modules
python3 -m venv .venv
. .venv/bin/activate
python -m pip install .
misp-modules -l 127.0.0.1 -p 6666
```

- Pin a reviewed revision for a reproducible workshop.
- For development, use an editable installation or Poetry.
- In production, use a service manager with the actual venv executable and service account.

No global sudo pip installation or rc.local startup is needed. See the README for the pinned lab setup.

Use a virtual environment and a supervised process



The slide shows the generic source workflow; the README adds the exact reviewed commit before installation. systemd does not source a user's shell profile or venv activation. Retain existing startup flags in service overrides. Remote/container deployments need an address reachable from MISP and an appropriate access boundary; do not expose the raw service to the Internet.

The service API is discoverable

Endpoint	What it tells you
`GET /healthcheck`	The HTTP service is responding.
`GET /version`	Installed package version.
`GET /modules`	Loaded module metadata and capabilities.
`GET /openapi.json`	Startup-generated API specification.
`GET /openapi`	Interactive Swagger UI using that specification.
`POST /query`	Dispatch a request to one selected module.
`GET /statistics`	Query counts, outcomes, latency and optional cache metrics.

A healthy service does not prove that a provider query will succeed.

The service API is discoverable
Endpoint What it tells you
GET /healthcheck The HTTP service is responding.
GET /version Installed package version.
GET /modules Loaded module metadata and capabilities.
GET /openapi.json Startup-generated API specification.
GET /openapi Interactive Swagger UI using that specification.
POST /query Dispatch a request to one selected module.
GET /statistics Query counts, outcomes, latency and optional cache metrics.
A healthy service does not prove that a provider query will succeed.

Restart after adding/removing modules to refresh discovery and OpenAPI. Swagger UI assets are referenced from a CDN; the JSON specification remains useful without those browser assets. The service API is independent of the standalone website and its login/-database. Core requests can succeed at HTTP level while returning a module error envelope.

Configure the right layer

- **MISP settings**: enablement, organisation restrictions and configuration sent in a query's `config`.
- **Module-server defaults**: optional `–module-config` JSON for credentials, URLs and other settings.
- **Service environment**: used by integrations that explicitly read environment variables, including some publication actions.
- **Website settings**: a separate application and stored module settings; not the MISP database.

Configure the right layer
- **MISP settings**: enablement, organisation restrictions and configuration sent in a query's `config`.
- **Module-server defaults**: optional `–module-config` JSON for credentials, URLs and other settings.
- **Service environment**: used by integrations that explicitly read environment variables, including some publication actions.
- **Website settings**: a separate application and stored module settings; not the MISP database.

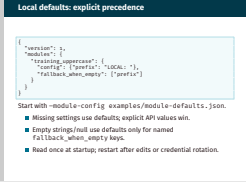
Do not assume that a setting in MISP's UI becomes an environment variable in a systemd service. Conversely, a module server's local defaults do not enable that module in MISP. Action modules may use params and environment variables; the generic config mechanism does not replace their documented contracts.

Local defaults: explicit precedence

```
{
  "version": 1,
  "modules": {
    "training_uppercase": {
      "config": {"prefix": "LOCAL: "},
      "fallback_when_empty": ["prefix"]
    }
  }
}
```

Start with `–module-config examples/module-defaults.json`.

- Missing settings use defaults; explicit API values win.
- Empty strings/null use defaults only for named `fallback_when_empty` keys.
- Read once at startup; restart after edits or credential rotation.



The file contains synthetic prefix text, not credentials. The lab verifies missing config, a nonempty override and empty/null fallback. False, zero, empty lists/objects are explicit values, not empty-string fallbacks. No recursive merging or environment-variable interpolation is performed. Any caller reaching the service can use its local defaults: separate trust boundaries may need separate module services.

Optional caching: choose read-only expansions

- Enable with `--cache-config`; disabled by default.
- Opt in each expansion module, with a TTL, entry limit and response-byte limit.
- Match the **whole effective request**, after local defaults: UUIDs, context, credentials and timeout matter.
- Identical concurrent requests can share one execution.
- Cache entries and counters are process-local and reset on restart.

Choose by behavior, not just family

Some expansion modules submit data or cause side effects. Caching one can suppress a repeated operation.

Optional caching: choose read-only expansions
Enable with --cache-config; disabled by default.
Opt in each expansion module, with a TTL, entry limit and response-byte limit.
Match the whole effective request, after local defaults: UUIDs, context, credentials and timeout matter.
Identical concurrent requests can share one execution.
Cache entries and counters are process-local and reset on restart.
Choose by behavior, not just family
Some expansion modules submit data or cause side effects. Caching one can suppress a repeated operation.

No Redis or external cache is required by the current optional cache. Import/export/action modules cannot be configured as loaded cached modules. Errors and late results are not retained; empty successful results can be. Hits do not extend expiry. Different attribute UUIDs deliberately do not share the same complete response. This is not a provider-wide rate limiter.

Observe queries, failures and cache savings

```
curl --fail --silent --show-error \
  http://127.0.0.1:6666/statistics | jq .
```

- Per-module and aggregate counts include loaded, unused modules.
- Distinguish module errors, exceptions and timeouts.
- Compare success/error rates, latency and in-flight requests.
- Optional cache metrics include hits, misses and coalesced queries.
- Query counts measure service requests, not provider HTTP calls.

Observe queries, failures and cache savings
curl --fail --silent --show-error \
http://127.0.0.1:6666/statistics | jq .
- Per-module and aggregate counts include loaded, unused modules.
- Distinguish module errors, exceptions and timeouts.
- Compare success/error rates, latency and in-flight requests.
- Optional cache metrics include hits, misses and coalesced queries.
- Query counts measure service requests, not provider HTTP calls.

Statistics are in memory per process and reset on restart. HTTP 200 with nonempty error/errors is a module failure, even with partial results. MISP supplies its own request timeout; direct service calls default to 300 seconds in this snapshot. A response timeout does not guarantee that the worker or remote operation stopped. Configure bounded provider-client timeouts in module code.

Operate the service within its trust boundary

- Keep the API reachable only by intended clients; it has no built-in authentication layer in the reviewed dispatcher.
- Review which modules contact providers or submit samples.
- Keep local credentials outside the checkout and readable only by the service account.
- Use normal debug logs first. `--debug-extended` logs outbound `requests` exchanges, including credentials and full payloads.

Operate the service within its trust boundary
- Keep the API reachable only by intended clients; it has no built-in authentication layer in the reviewed dispatcher.
- Review which modules contact providers or submit samples.
- Keep local credentials outside the checkout and readable only by the service account.
- Use normal debug logs first. `--debug-extended` logs outbound `requests` exchanges, including credentials and full payloads.

This is an operational consequence of the features, not a claim that every module has identical secret handling. Extended tracing is unredacted and only covers requests and SDKs using its standard adapter, not every HTTP library. Direct Python handler calls bypass dispatcher defaults, cache and statistics. A website admin login does not add authentication to the raw API.

Troubleshoot the failing layer

Symptom	First checks
Cannot reach service	Process, listen address, container network, host/port.
Module absent	Correct venv/package, optional dependencies, loader logs, restart.
Present, absent in MISP	Correct family settings, metadata type, reload settings.
Visible, unavailable to user	Module enablement, organisation restriction, supported input.
HTTP 200, failed query	Inspect `error`/`errors`; config, quotas, provider response.
Unexpected stale result	Effective request, TTL, correct process and cache configuration.

Troubleshoot the failing layer

Symptom	First checks
Cannot reach service	Process, listen address, container network, host/port.
Module absent	Correct venv/package, optional dependencies, loader logs, restart.
Present, absent in MISP	Correct family settings, metadata type, reload settings.
Visible, unavailable to user	Module enablement, organisation restriction, supported input.
HTTP 200, failed query	Inspect `error`/`errors`; config, quotas, provider response.
Unexpected stale result	Effective request, TTL, correct process and cache configuration.

Work from service health to discovery to module execution to UI. Run connectivity checks inside the MISP host/container, not only on the module server. Syntax errors can prevent startup; missing optional imports are often skipped. A local source edit is ineffective if the running service uses a different installed package or never restarted.

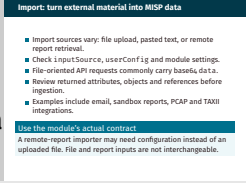
Import, export and act

Import: turn external material into MISP data

- Import sources vary: file upload, pasted text, or remote report retrieval.
- Check `inputSource`, `userConfig` and module settings.
- File-oriented API requests commonly carry base64 `data`.
- Review returned attributes, objects and references before ingestion.
- Examples include email, sandbox reports, PCAP and TAXII integrations.

Use the module's actual contract

A remote-report importer may need configuration instead of an uploaded file. File and report inputs are not interchangeable.

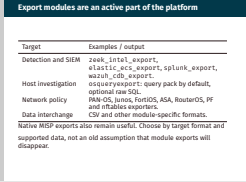


Do not treat the testimport teaching stub as a production CSV parser. The standalone website now supports active import modules, including upload/paste and remote-report forms. Uploaded inputs are not saved for replay there. Review per-module schemas and external effects before using real reports.

Export modules are an active part of the platform

Target	Examples / output
Detection and SIEM	`zeek_intel_export`, `elastic_ecs_export`, `splunk_export`, `wazuh_cdb_export`.
Host investigation	`osqueryexport`: query pack by default, optional raw SQL.
Network policy	PAN-OS, Junos, FortiOS, ASA, RouterOS, PF and nftables exporters.
Data interchange	CSV and other module-specific formats.

Native MISP exports also remain useful. Choose by target format and supported data, not an old assumption that module exports will disappear.



The current source has 32 export module files. Event collections and object attributes are supported by several newer exporters; some exporters still require a single event. Check each module's contract. Network ACL fragments are vendor-specific: a generic rule is not portable across devices, and an exported file has not been activated.

Lab: generate a Zeek Intel file

Use the included `examples/export-request.json`.

```
curl --fail --silent --show-error \
  http://127.0.0.1:6666/query \
  -H 'Content-Type: application/json' \
  --data @examples/export-request.json > /tmp/zeek-response.json

python3 -c 'import base64,json; r=json.load(open("/tmp/zeek-
    response.json")); print(base64.b64decode(r["data"]).decode("
    ascii"))'
```

Expected: `example.org` and `192.0.2.10`, with native Zeek types. The non-IDS attribute is excluded by default.



The response envelope's data is base64, not an event JSON document. In automation, check error/errors before decoding data. The functional lab verifies the #fields header, Intel::DOMAIN/ADDR and non-IDS exclusion. No Zeek service is contacted or reloaded. Distribution, policy loading and operational detection checks remain separate.

Exporting a file and publishing an action differ

	Export	Workflow action
Main result	A file or generated representation.	An operation and its status/result.
Examples	Zeek feed, osquery pack, firewall fragment.	Notification, local pack write, SIEM/EDR publication.
Credentials	Often unnecessary for local generation.	Often needed for the target; some support credential-free dry runs.
Completion	Output generated.	Check acknowledgement, partial failures and activation requirements.

Exporting a file and publishing an action differ

	Export	Workflow action
Main result	A file or generated representation.	An operation and its status/result.
Examples	Zeek feed, osquery pack, firewall fragment.	Notification, local pack write, SIEM/EDR publication.
Credentials	Often unnecessary for local generation.	Often needed for the target; some support credential-free dry runs.
Completion	Output generated.	Check acknowledgement, partial failures and activation requirements.

Neither family is guaranteed side-effect-free: some exports contact services, and some expansions submit data. Classify actual behavior. An action can publish to an endpoint, or only write a local artifact. Do not equate a successful export, local publication, target acknowledgement and effective detection on endpoints.

Workflow actions connect intelligence to operations

- Notifications: Slack, Matrix, IRC, Signal and Nextcloud Talk.
- Target publication: CrowdStrike IOC, SentinelOne IOC, QRadar reference sets and Wazuh CDB lists.
- Local artifact publication: `osquery_pack_publish`.
- Inspect action metadata: parameters, blocking behavior, filters and expected MISP-core format.
- Supply the intended data selection and explicit target scope.

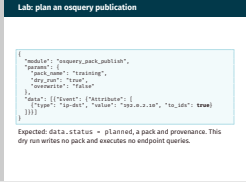
Workflow actions connect intelligence to operations
- Notifications: Slack, Matrix, IRC, Signal and Nextcloud Talk.
- Target publication: CrowdStrike IOC, SentinelOne IOC, QRadar reference sets and Wazuh CDB lists.
- Local artifact publication: osquery_pack_publish.
- Inspect action metadata: parameters, blocking behavior, filters and expected MISP-core format.
- Supply the intended data selection and explicit target scope.

Action request parameters usually live in params, not the expansion config object. Individual actions can also require credentials in the service environment. Some actions do not support workflow filteredItems/matchingData; do not assume every action consumes an identical workflow payload. Blocking action failures can stop a workflow path; inspect its returned data and module-specific errors, including partial publication failures.

Lab: plan an osquery publication

```
{
  "module": "osquery_pack_publish",
  "params": {
    "pack_name": "training",
    "dry_run": "true",
    "overwrite": "false"
  },
  "data": [{"Event": {"Attribute": [
    {"type": "ip-dst", "value": "192.0.2.10", "to_ids": true}
  ]}}]
}
```

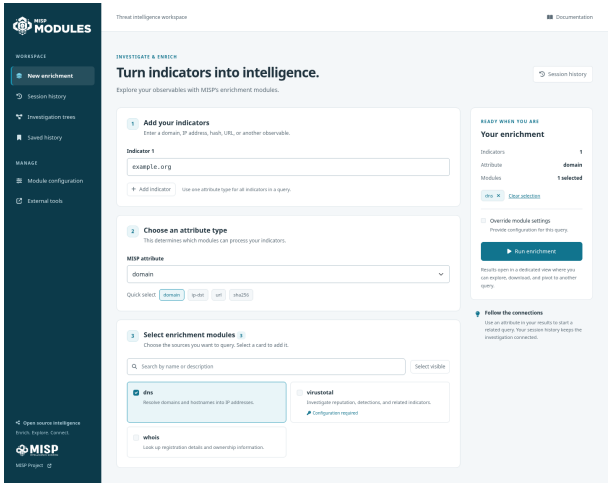
Expected: `data.status = planned`, a pack and provenance. This dry run writes no pack and executes no endpoint queries.



The included API verification asserts planned rather than published. Real local publication needs OSQUERY_PACK_DIRECTORY on the module server. Existing packs require explicit overwrite, which replaces the complete snapshot. The action does not implement a generic push API to an osquery agent: distribute/reference the pack and reload configuration separately. Other actions have different defaults; request dry_run explicitly where supported.

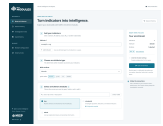
The standalone website

An enrichment workspace without a MISP instance



Current upstream documentation screenshot; sample catalogue from browser fixtures.

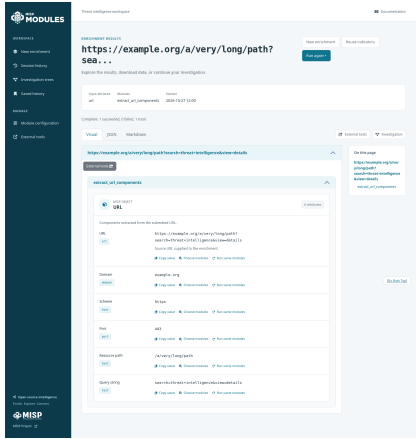
An enrichment workspace without a MISP instance



Current upstream documentation screenshot; sample catalogue from browser fixtures.

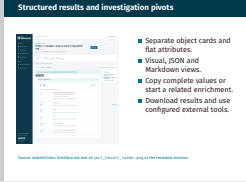
Choose the attribute type, compatible modules and indicators before submitting. Changing attribute type clears incompatible selections. Per-query overrides are separate from saved module settings. The website is a client of the same module service, not a MISP event database. This screenshot is upstream's illustrative browser-test capture, not a live provider test.

Structured results and investigation pivots



- Separate object cards and flat attributes.
- Visual, JSON and Markdown views.
- Copy complete values or start a related enrichment.
- Download results and use configured external tools.

Source: website/doc/interface.md and `object_result_cards.png` at the reviewed revision.



The old deck called external tools still in development. The current application implements configured external-tool flows; availability depends on configured destinations and their keys. A current migration repairs legacy external_tools.api_key schema state. Rendering and a copy action do not establish provider availability or publication success.

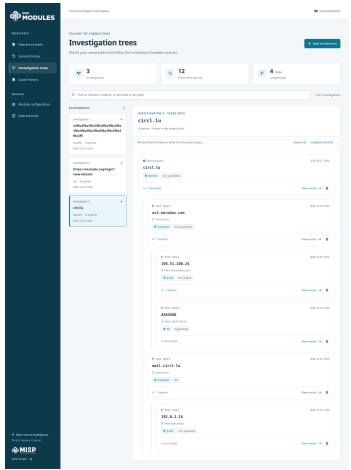
Import and export are website workflows too

- **Import data**: choose an active module and provide a supported upload, paste or remote-report configuration.
- **Export data**: provide MISP JSON, or choose a CIRCL OSINT feed event, then configure an active exporter.
- Results, text previews, decoded downloads and history use the shared interface.
- With `ADMIN_PASSWORD` configured, exports and their saved results require an administrator login.

Import and export are website workflows too
- **Import data**: choose an active module and provide a supported upload, paste or remote-report configuration.
- **Export data**: provide MISP JSON, or choose a CIRCL OSINT feed event, then configure an active exporter.
- Results, text previews, decoded downloads and history use the shared interface.
- With ADMIN_PASSWORD configured, exports and their saved results require an administrator login.

The reviewed website bounds uploads to 20 MiB. It accepts several event envelope shapes but honors modules that require a single event. Feed access occurs on the website server, not in the browser. Uploaded import/export input is not retained for replay; provide it again on a new run. The raw service and website authentication boundaries are separate.

Keep the investigation path



Session history, saved trees and related-query paths in the current upstream interface.



Follow parent and child queries, search paths, and keep context for pivots. The website can save session paths as investigation trees. For a website update, back up its database and apply shipped migrations with poetry run mmw db upgrade before restart. db init is for initial setup; db migrate generates a migration for developers, it does not apply shipped updates. The website and its production deployment were not exercised by the API-only lab.

Develop and verify a module

A small interface, a real contract

- `introspection()`: supported input/output or import/export setup.
- `version()`: metadata, capabilities and configuration schema.
- `dict_handler(request=...)`: dictionary execution path supported by the current dispatcher.
- `handler(q=...)`: legacy JSON interface, still supported.
- Return useful `error` messages and valid result envelopes.

Choose the result format deliberately

The lab's simple text module uses legacy results. For structured enrichment, start from a current `misp_standard` module and retain source UUIDs and references.

A small interface, a real contract
introspection(): supported input/output or import/export setup.
version(): metadata, capabilities and configuration schema.
dict_handler(request=...): dictionary execution path supported by the current dispatcher.
handler(q=...): legacy JSON interface, still supported.
Return useful error messages and valid result envelopes.
Choose the result format deliberately
The lab's simple text module uses legacy results. For structured enrichment, start from a current misp_standard module and retain source UUIDs and references.

The dispatcher tries dict_handler first and falls back to handler. The supplied module implements both so their outputs can be compared. Avoid raising AttributeError accidentally within a dict_handler; the current dispatcher's fallback also catches that exception. Do not copy the old deck's truncated DNS implementation with an except ellipsis.

Metadata is part of discovery

```
moduleinfo = {
    "version": "1.0",
    "author": "MISP Project training contributors",
    "name": "Training uppercase",
    "description": "Uppercase synthetic training text without
    network access.",
    "module-type": ["expansion", "hover"],
}
moduleconfig = ["prefix"]
mispattributes = {"input": ["text"], "output": ["text"]}
```

`training_uppercase.py` gives the API identifier. The display name is *Training uppercase*.

Metadata is part of discovery
moduleinfo = { "version": "1.0", "author": "MISP Project training contributors", "name": "Training uppercase", "description": "Uppercase synthetic training text without network access.", "module-type": ["expansion", "hover"], }
moduleconfig = ["prefix"]
mispattributes = {"input": ["text"], "output": ["text"]}
training_uppercase.py gives the API identifier. The display name is Training uppercase.

The module is a local-only contract demonstration. Use the filename as the API identifier and keep the display name descriptive. Returning metadata does not install dependencies or declare the module enabled for every organisation in MISP.

A complete handler, with validation

```
def dict_handler(request):
    if not isinstance(request, dict):
        return {"error": "Expected a request object."}
    value = request.get("text")
    if not isinstance(value, str) or not value.strip():
        return {"error": "A nonempty text value is required."}
    config = request.get("config", {})
    if not isinstance(config, dict):
        return {"error": "config must be an object."}
    prefix = config.get("prefix", "")
    if not isinstance(prefix, str):
        return {"error": "prefix must be a string."}
    return {"results": [{"types": ["text"], "values": [prefix +
    value.upper()]}]}
```


A complete handler, with validation

All lines are taken from the runnable file, not pseudocode. Show handler(q) below it in the source: it deserializes JSON and delegates here. The module is a contract demonstration, not a production enrichment source. It validates a nonempty text value and the prefix/config types before returning. Production modules also need bounded I/O and tests for dependency/provider failures.

Load a local module without modifying the package

```
examples/custom/
  expansion/
    training_uppercase.py
```

```
misp-modules -l 127.0.0.1 \
  --custom "$PWD/examples/custom"
```

- Pass the custom *root*, not its `expansion` child.
- The loader discovers valid `.py` files; no `__all__` edit is needed.
- Restart, check logs and `/modules`, then enable/configure it in MISP.

Load a local module without modifying the package
examples/custom/
expansion/
training_uppercase.py
misp-modules -l 127.0.0.1 \
--custom "$PWD/examples/custom"
Pass the custom root, not its expansion child.
The loader discovers valid .py files; no __all__ edit is needed.
Restart, check logs and /modules, then enable/configure it in MISP.

Run from this deck's directory. For the complete lab retain all README flags, including defaults and caching. Custom names must be unique: a custom module with the same identifier replaces an installed module. The service user needs read/traverse access. A non-editable pip installation uses an installed copy, so editing an unrelated checkout does not change a running service.

Test behavior across the service boundary

- Compare dictionary and legacy handlers on valid and invalid input.
- Verify discovery and generated OpenAPI metadata.
- Check structured output and references using a local parser.
- Decode an export and verify indicator selection.
- Assert action status is `planned` in the dry run.
- Exercise defaults, API overrides, empty-value fallback, cache hits and error reporting.

The included `examples/verify_lab.py` performs 12 functional/contract tests.

Test behavior across the service boundary
- Compare dictionary and legacy handlers on valid and invalid input.
- Verify discovery and generated OpenAPI metadata.
- Check structured output and references using a local parser.
- Decode an export and verify indicator selection.
- Assert action status is planned in the dry run.
- Exercise defaults, API overrides, empty-value fallback, cache hits and error reporting.

The included examples/verify_lab.py performs 12 functional/contract tests.

The tests fail if expected modules or startup options are absent. They do not contact external enrichment providers or perform a real publication. This scope does not validate MISP UI behavior, the website server, or vendor credentials. Direct handler tests alone cannot validate dispatcher caching or defaults.

Workshop: discover, transform, verify

1. **Discover**: find the lab modules and their formats in `/modules` and `/openapi.json`.
2. **Transform**: parse a URL, decode a Zeek file and inspect an osquery publication plan.
3. **Develop**: change the custom module's prefix and test a request override; inspect cache metrics and error responses.

```
python3 examples/verify_lab.py \
  --url http://127.0.0.1:6666 -v
```

Follow the README's full startup command before running the verifier.



Suggested timing: 10 minutes for discovery and structured enrichment, 10 minutes for export/action interpretation, and 15–25 minutes for customization. Ask participants why a repeated UUID affects cache matching, why HTTP 200 need not mean success, and why a planned pack is not an endpoint detection. The trainer guide includes expected answers and the limits of the lab.

What to take into the next integration

1. Discover the **running** module set and its contracts.
2. Separate service configuration, MISP enablement and provider readiness.
3. Preserve evidence and review enrichment before operational use.
4. Distinguish file generation, publication and activation.
5. Test through the dispatcher when relying on its configuration or cache.

Keep the material current

Recheck the pinned source audit, exercise results and module-specific documentation before teaching a newer revision.

What to take into the next integration
1. Discover the running module set and its contracts.
2. Separate service configuration, MISP enablement and provider readiness.
3. Preserve evidence and review enrichment before operational use.
4. Distinguish file generation, publication and activation.
5. Test through the dispatcher when relying on its configuration or cache.
Keep the material current
Recheck the pinned source audit, exercise results and module-specific documentation before teaching a newer revision.

Invite participants to propose a concrete integration and identify its family, input/output, source behavior, credentials, failure cases and validation plan. This replaces the old speculative roadmap with current, demonstrable capabilities.

References and next steps

- MISP/misp-modules: code and current catalogue
- Module documentation: install, contribute and module guides
- Standalone website and update instructions
- Standalone CLI client
- This deck's `README.md`: pinned sources, trainer notes, reproducible lab and validation scope.
- Continue with *Contributing MISP modules with an AI agent*; see the companion `a.14-ai-assisted-misp-modules` deck.

References and next steps
- MISP/misp-modules: code and current catalogue
- Module documentation: install, contribute and module guides
- Standalone website and update instructions
- Standalone CLI client
- This deck's README.md: pinned sources, trainer notes, reproducible lab and validation scope.
- Continue with Contributing MISP modules with an AI agent; see the companion a.14-ai-assisted-misp-modules deck.

The README source audit uses immutable revision links for the reviewed behavior. The slide links deliberately lead to current project entry points. Use actual service discovery rather than treating a documentation catalogue as a guarantee that all modules are loaded or enabled.